



Threagile

Agile Threat Modeling

Threat Model Report

Notes App - Amenazas (TP14)

5 September 2026

Equipo DevOps - Notes App

Table of Contents

Results Overview

Management Summary	4
Impact Analysis of 42 Initial Risks in 17 Categories	5
Risk Mitigation	7
Impact Analysis of 42 Remaining Risks in 17 Categories	8
Application Overview	10
Data-Flow Diagram	11
Security Requirements	13
Abuse Cases	14
Tag Listing	15
STRIDE Classification of Identified Risks	16
Assignment by Function	19
RAA Analysis	22
Data Mapping	24
Out-of-Scope Assets: 0 Assets	25
Potential Model Failures: 2 / 2 Risks	26
Questions: 0 / 0 Questions	27

Risks by Vulnerability Category

Identified Risks by Vulnerability Category	28
Path-Traversal: 1 / 1 Risk	29
SQL/NoSQL-Injection: 1 / 1 Risk	31
Cross-Site Scripting (XSS): 2 / 2 Risks	33
Missing Authentication: 1 / 1 Risk	35
Missing Hardening: 3 / 3 Risks	37
Accidental Secret Leak: 1 / 1 Risk	39
Code Backdooring: 2 / 2 Risks	41
Container Base Image Backdooring: 10 / 10 Risks	44
Cross-Site Request Forgery (CSRF): 2 / 2 Risks	47
Missing Build Infrastructure: 1 / 1 Risk	49
Missing Vault (Secret Storage): 1 / 1 Risk	51
Server-Side Request Forgery (SSRF): 6 / 6 Risks	53
Unchecked Deployment: 2 / 2 Risks	55
Unencrypted Communication: 1 / 1 Risk	57
Unencrypted Technical Assets: 5 / 5 Risks	59
Unguarded Access From Internet: 1 / 1 Risk	61
Missing Network Segmentation: 2 / 2 Risks	63

Risks by Technical Asset

Identified Risks by Technical Asset	65
backend: 7 / 7 Risks	66
postgres: 5 / 5 Risks	70
github-actions-pipeline: 5 / 5 Risks	73
kubernetes-api-server: 4 / 4 Risks	76
nginx: 3 / 3 Risks	79
volumen-postgres: 3 / 3 Risks	81
cadvisor: 1 / 1 Risk	83
docker-registry: 3 / 3 Risks	85
grafana: 3 / 3 Risks	87
ingress-controller: 1 / 1 Risk	89
node-exporter: 1 / 1 Risk	92
prometheus: 6 / 6 Risks	94
cliente-web: 0 / 0 Risks	99

Data Breach Probabilities by Data Asset

Identified Data Breach Probabilities by Data Asset	101
contenido-notas: 23 / 23 Risks	102
credenciales-bd: 19 / 19 Risks	104
credenciales-registro: 8 / 8 Risks	105
credenciales-usuario: 3 / 3 Risks	106
metadata-descubrimiento-k8s: 10 / 10 Risks	107
metricas-monitoreo: 11 / 11 Risks	108
token-service-account-prometheus: 7 / 7 Risks	109

Trust Boundaries

cluster-kubernetes	110
--------------------	-----

About Threagile

Risk Rules Checked by Threagile	112
Disclaimer	125

Management Summary

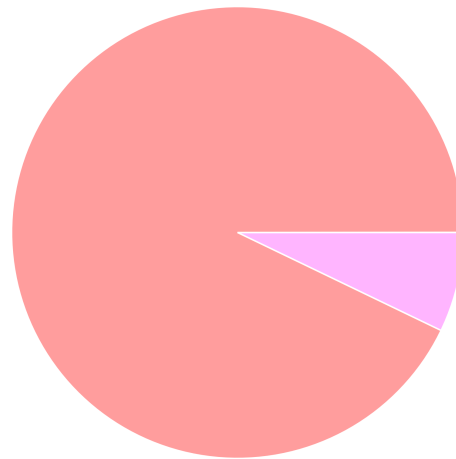
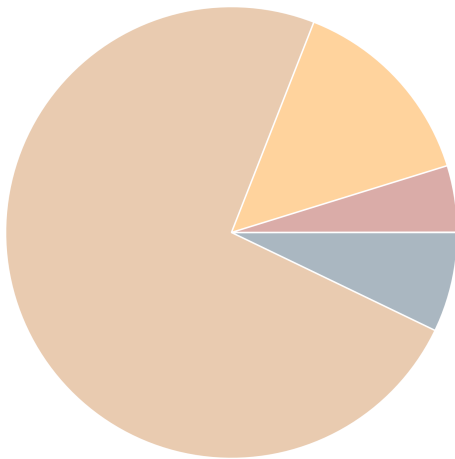
Threagile toolkit was used to model the architecture of "Notes App - Amenazas (TP14)" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "Notes App - Amenazas (TP14)" whether the mitigation advices have been applied or not.

Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **42 initial risks** in **17 categories** have been identified during the threat modeling process:

0 critical risk
2 high risk
6 elevated risk
31 medium risk
3 low risk

39 unchecked
0 in discussion
3 accepted
0 in progress
0 mitigated
0 false positive



Impact Analysis of 42 Initial Risks in 17 Categories

The most prevalent impacts of the **42 initial risks** (distributed over **17 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

High: Path-Traversal: 1 Initial Risk - Exploitation likelihood is *Very Likely* with *High* impact.

If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

High: SQL/NoSQL-Injection: 1 Initial Risk - Exploitation likelihood is *Very Likely* with *High* impact.

If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: Cross-Site Scripting (XSS): 2 Initial Risks - Exploitation likelihood is *Likely* with *High* impact.

If this risk remains unmitigated, attackers might be able to access individual victim sessions and steal or modify user data.

Elevated: Missing Authentication: 1 Initial Risk - Exploitation likelihood is *Likely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: Missing Hardening: 3 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.

If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Medium: Accidental Secret Leak: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk is unmitigated, attackers which have access to affected sourcecode repositories or artifact registries might find secrets accidentally checked-in.

Medium: Code Backdooring: 2 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk remains unmitigated, attackers might be able to execute code on and completely takeover production environments.

Medium: Container Base Image Backdooring: 10 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Medium: Cross-Site Request Forgery (CSRF): 2 Initial Risks - Exploitation likelihood is *Very Likely* with *Low* impact.

If this risk remains unmitigated, attackers might be able to trick logged-in victim users into unwanted actions within the web application by visiting an attacker controlled web site.

Medium: Missing Build Infrastructure: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: Missing Vault (Secret Storage): 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: Server-Side Request Forgery (SSRF): 6 Initial Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Medium: Unchecked Deployment: 2 Initial Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Medium: Unencrypted Communication: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk is unmitigated, network attackers might be able to eavesdrop on unencrypted sensitive data sent between components.

Medium: Unencrypted Technical Assets: 5 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Medium: Unguarded Access From Internet: 1 Initial Risk - Exploitation likelihood is *Very Likely* with *Low* impact.

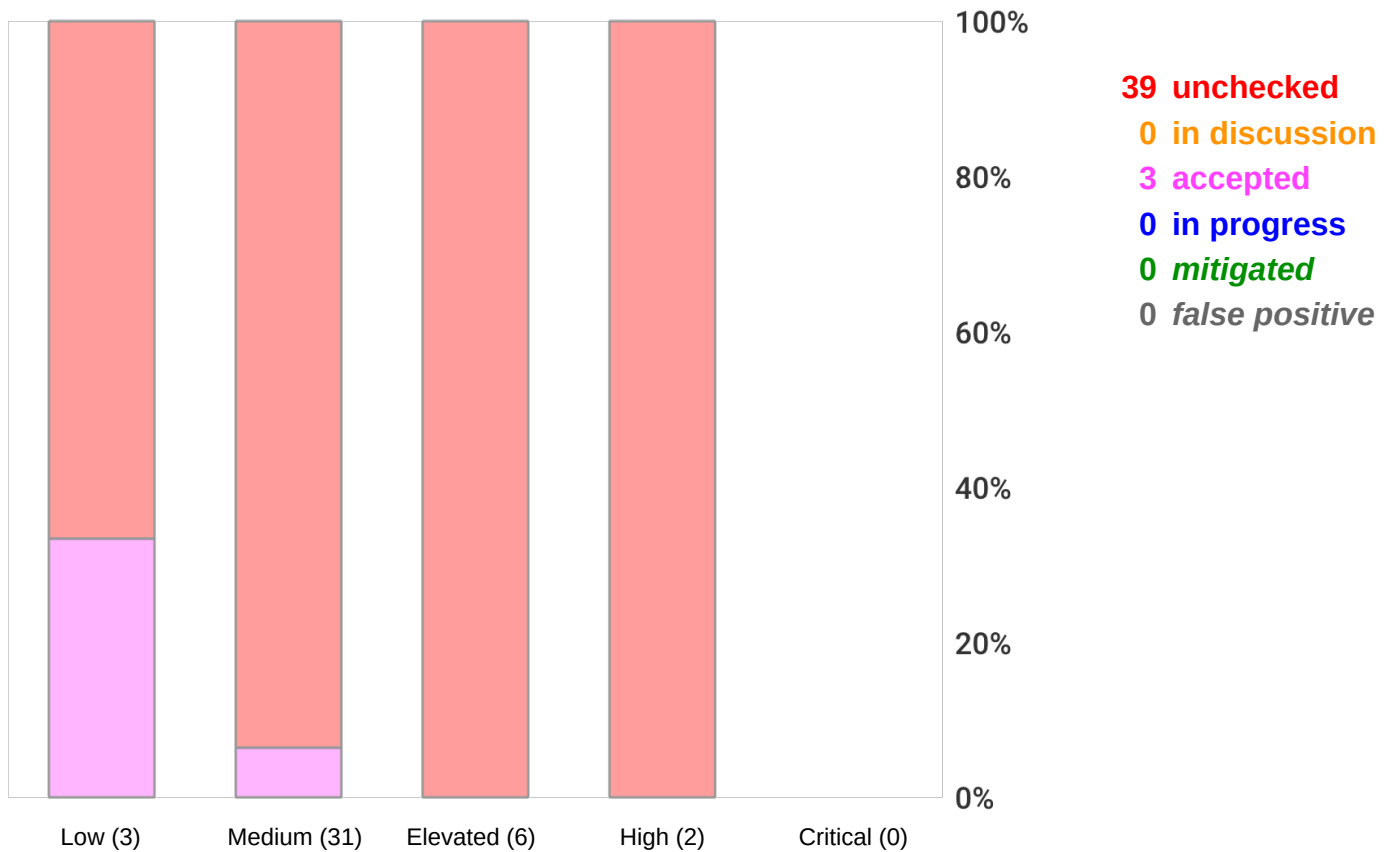
If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Low: Missing Network Segmentation: 2 Initial Risks - Exploitation likelihood is *Unlikely* with *Low* impact.

If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are not separated by network segmentation.

Risk Mitigation

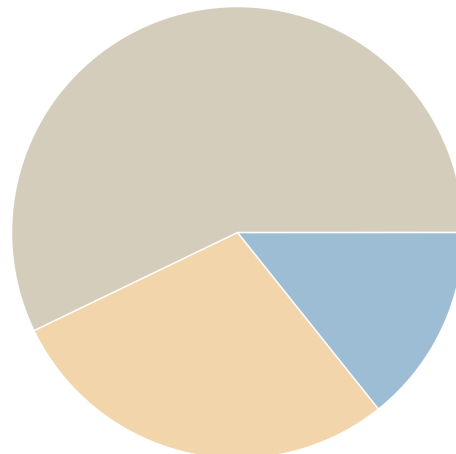
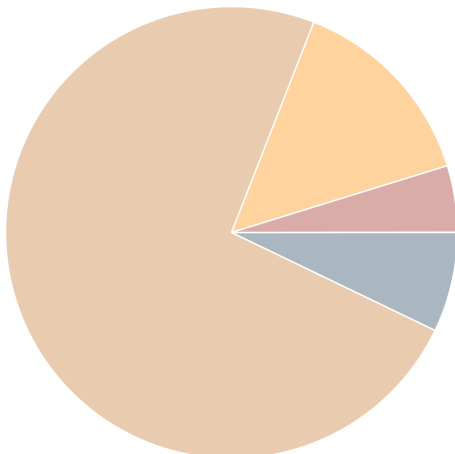
The following chart gives a high-level overview of the risk tracking status (including mitigated risks):



After removal of risks with status *mitigated* and *false positive* the following 42 remain unmitigated:

0 unmitigated critical risk
2 unmitigated high risk
6 unmitigated elevated risk
31 unmitigated medium risk
3 unmitigated low risk

0 business side related
6 architecture related
12 development related
24 operations related



Impact Analysis of 42 Remaining Risks in 17 Categories

The most prevalent impacts of the **42 remaining risks** (distributed over **17 risk categories**) are (taking the severity ratings into account and using the highest for each category):

Risk finding paragraphs are clickable and link to the corresponding chapter.

High: Path-Traversal: 1 Remaining Risk - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

High: SQL/NoSQL-Injection: 1 Remaining Risk - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: Cross-Site Scripting (XSS): 2 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk remains unmitigated, attackers might be able to access individual victim sessions and steal or modify user data.

Elevated: Missing Authentication: 1 Remaining Risk - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: Missing Hardening: 3 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Medium: Accidental Secret Leak: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers which have access to affected sourcecode repositories or artifact registries might find secrets accidentally checked-in.

Medium: Code Backdooring: 2 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk remains unmitigated, attackers might be able to execute code on and completely takeover production environments.

Medium: Container Base Image Backdooring: 10 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Medium: Cross-Site Request Forgery (CSRF): 2 Remaining Risks - Exploitation likelihood is *Very Likely* with *Low* impact.

If this risk remains unmitigated, attackers might be able to trick logged-in victim users into unwanted actions within the web application by visiting an attacker controlled web site.

Medium: Missing Build Infrastructure: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: Missing Vault (Secret Storage): 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: Server-Side Request Forgery (SSRF): 6 Remaining Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Medium: Unchecked Deployment: 2 Remaining Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Medium: Unencrypted Communication: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk is unmitigated, network attackers might be able to eavesdrop on unencrypted sensitive data sent between components.

Medium: Unencrypted Technical Assets: 5 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.

If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Medium: Unguarded Access From Internet: 1 Remaining Risk - Exploitation likelihood is *Very Likely* with *Low* impact.

If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Low: Missing Network Segmentation: 2 Remaining Risks - Exploitation likelihood is *Unlikely* with *Low* impact.

If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are not separated by network segmentation.

Application Overview

Business Criticality

The overall business criticality of "Notes App - Amenazas (TP14)" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

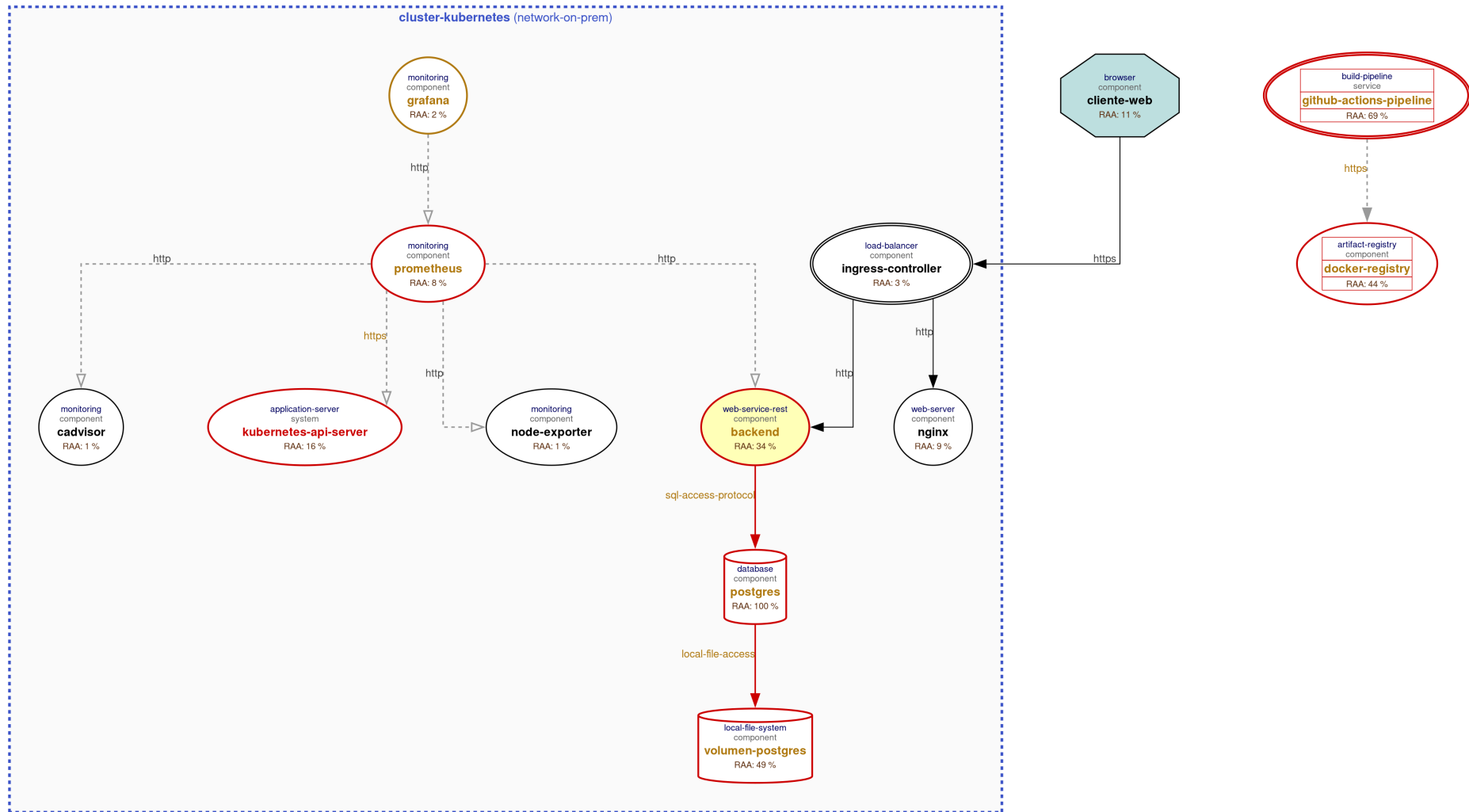
Business Overview

Technical Overview

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.

Data-Flow Diagram - Notes App - Amenazas (TP14)



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **42 potential risks** have been identified during the threat modeling process of which **2 in the Spoofing** category, **21 in the Tampering** category, **0 in the Repudiation** category, **15 in the Information Disclosure** category, **0 in the Denial of Service** category, and **4 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

Medium: **Cross-Site Request Forgery (CSRF)**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *Low* impact.

When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.

Tampering

High: **SQL/NoSQL-Injection**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *High* impact.

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed or stored.

Elevated: **Cross-Site Scripting (XSS)**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *High* impact.

For each web application Cross-Site Scripting (XSS) risks might arise. In terms of the overall risk level take other applications running on the same domain into account as well.

Elevated: **Missing Hardening**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Medium: **Code Backdooring**: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

For each build-pipeline component Code Backdooring risks might arise where attackers compromise the build-pipeline in order to let backdoored artifacts be shipped into production. Aside from direct code backdooring this includes backdooring of dependencies and even of more lower-level build infrastructure, like backdooring compilers (similar to what the XcodeGhost malware did) or dependencies.

Medium: **Container Base Image Backdooring**: 10 / 10 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

Medium: Missing Build Infrastructure: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: Unchecked Deployment: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Repudiation

n/a

Information Disclosure

High: Path-Traversal: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *High* impact.

When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed or stored.

Medium: Accidental Secret Leak: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *High* impact. Sourcecode repositories (including their histories) as well as artifact registries can accidentally contain secrets like checked-in or packaged-in passwords, API tokens, certificates, crypto keys, etc.

Medium: Missing Vault (Secret Storage): 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Medium: Server-Side Request Forgery (SSRF): 6 / 6 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Medium: **Unencrypted Communication**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *High* impact.

Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Medium: **Unencrypted Technical Assets**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

Due to the confidentiality rating of the technical asset itself and/or the processed data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Denial of Service

n/a

Elevation of Privilege

Elevated: **Missing Authentication**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes or stores sensitive data.

Medium: **Unguarded Access From Internet**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *Low* impact.

Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Low: **Missing Network Segmentation**: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.

Highly sensitive assets and/or datastores residing in the same network segment than other lower sensitive assets (like web servers or content management systems etc.) should be better protected by a network segmentation trust-boundary.

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **42 potential risks** have been identified during the threat modeling process of which **0 should be checked by Business Side**, **6 should be checked by Architecture**, **12 should be checked by Development**, and **24 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

n/a

Architecture

Elevated: **Missing Authentication**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *Medium* impact.

Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Include the build infrastructure in the model.

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

Medium: **Unchecked Deployment**: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

Apply DevSecOps best-practices and use scanning tools to identify vulnerabilities in source- or byte-code, dependencies, container layers, and optionally also via dynamic scans against running test systems.

Medium: **Unguarded Access From Internet**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *Low* impact.

Encapsulate the asset behind a guarding service, application, or reverse-proxy. For admin maintenance a bastion-host should be used as a jump-server. For file transfer a store-and-forward-host should be used as an indirect file exchange platform.

Development

High: Path-Traversal: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *High* impact.

Before accessing the file cross-check that it resides in the expected folder and is of the expected type and filename/suffix. Try to use a mapping if possible instead of directly accessing by a filename which is (partly or fully) provided by the caller. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

High: SQL/NoSQL-Injection: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *High* impact.

Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Elevated: Cross-Site Scripting (XSS): 2 / 2 Risks - Exploitation likelihood is *Likely* with *High* impact.

Try to encode all values sent back to the browser and also handle DOM-manipulations in a safe way to avoid DOM-based XSS. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Medium: Cross-Site Request Forgery (CSRF): 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *Low* impact.

Try to use anti-CSRF tokens or the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Medium: Server-Side Request Forgery (SSRF): 6 / 6 Risks - Exploitation likelihood is *Unlikely* with *Medium* impact.

Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Operations

Elevated: Missing Hardening: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

Medium: Accidental Secret Leak: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *High* impact.

Establish measures preventing accidental check-in or package-in of secrets into sourcecode repositories and artifact registries. This starts by using good .gitignore and .dockerignore files, but does not stop there. See for example tools like "*git-secrets*" or "*Talisman*" to have check-in preventive measures for secrets. Consider also to regularly scan your repositories for secrets accidentally checked-in using scanning tools like "*gitLeaks*" or "*gitrob*".

Medium: Code Backdooring: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

Reduce the attack surface of backdooring the build pipeline by not directly exposing the build pipeline components on the public internet and also not exposing it in front of unmanaged (out-of-scope) developer clients. Also consider the use of code signing to prevent code modifications.

Medium: Container Base Image Backdooring: 10 / 10 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

Apply hardening of all container infrastructures (see for example the *CIS-Benchmarks for Docker and Kubernetes* and the *Docker Bench for Security*). Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's *Distroless* base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.

Medium: Unencrypted Communication: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *High* impact.

Apply transport layer encryption to the communication link.

Medium: Unencrypted Technical Assets: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

Apply encryption to the technical asset.

Low: Missing Network Segmentation: 2 / 2 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.

Apply a network segmentation trust-boundary around the highly sensitive assets and/or datastores.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

postgres: RAA 100%

Base de datos relacional PostgreSQL encargada de resguardar los datos de las notas de forma persistente.

github-actions-pipeline: RAA 69%

Runner externo que construye y publica en Docker Hub la imagen del backend histórico desde main o develop.

volumen-postgres: RAA 49%

PersistentVolumeClaim de Kubernetes donde PostgreSQL guarda físicamente los archivos de la base de datos.

docker-registry: RAA 44%

Registro público donde el pipeline de CI/CD publica la imagen del backend.

backend: RAA 34%

Servidor de aplicación que gestiona la lógica de las notas y expone métricas internas para Prometheus.

kubernetes-api-server: RAA 16%

Plano de control del clúster. Prometheus lo consulta para el service discovery dinámico (kubernetes_sd_configs) en vez de usar hostnames fijos, autenticado con el token de la ServiceAccount "prometheus-sa" y autorizado por un ClusterRole de solo lectura (pods, services, endpoints, nodes).

cliente-web: RAA 11%

El cliente HTTP (navegador) en el dispositivo del usuario que inicia el acceso a la Notes App a través de internet de forma pública.

nginx: RAA 9%

Pod Nginx que sirve el frontend estático de la Notes App detrás del Ingress.

prometheus: RAA 8%

Sistema de monitoreo que recopila (scrapes) las métricas de rendimiento del ecosistema de la Notes App, y descubre sus targets (backend, node-exporter, cadvisor) de forma dinámica consultando al API Server de Kubernetes por label, en vez de usar hostnames fijos.

ingress-controller: RAA 3%

Ingress de Kubernetes (ingress-nginx) que termina TLS y enruta por path hacia los Services internos - "/" al frontend, "/api" y "/health" directo al backend - según el chart Helm de TP10.

grafana: RAA 2%

Plataforma web interactiva para visualizar dashboards basados en las métricas consolidadas.

cadvisor: RAA 1%

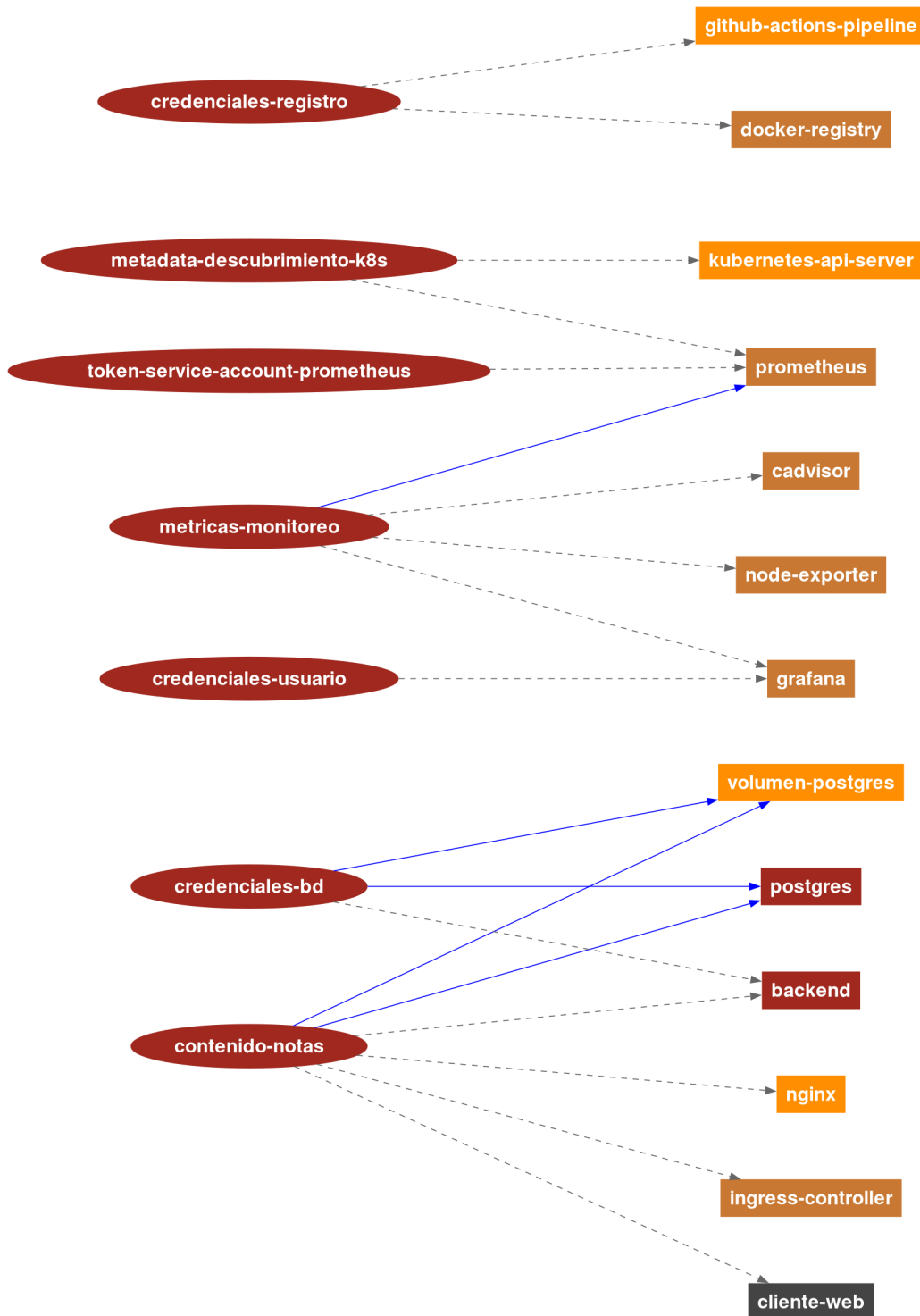
DaemonSet encargado de exponer de forma pasiva las métricas de rendimiento internas de los contenedores del clúster.

node-exporter: RAA 1%

DaemonSet que expone de forma pasiva las métricas del hardware de cada nodo del clúster.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 2 / 2 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium: Missing Build Infrastructure: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: Missing Vault (Secret Storage): 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability Category

In total **42 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 2 as high, 6 as elevated, 31 as medium, and 3 as low.**

These risks are distributed across **17 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

Path-Traversal: 1 / 1 Risk

Description (Information Disclosure): [CWE 22](#)

When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed or stored.

Impact

If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Detection Logic

Filesystems accessed by in-scope callers.

Risk Rating

The risk rating depends on the sensitivity of the data stored inside the technical asset.

False Positives

File accesses by filenames not consisting of parts controllable by the caller can be considered as false positives after individual review.

Mitigation (Development): Path-Traversal Prevention

Before accessing the file cross-check that it resides in the expected folder and is of the expected type and filename/suffix. Try to use a mapping if possible instead of directly accessing by a filename which is (partly or fully) provided by the caller. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V12 - File and Resources Verification Requirements](#)

Cheat Sheet: [Input Validation Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Path-Traversal** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

Path-Traversal risk at **postgres** against filesystem **volumen-postgres** via **escritura_volumen**: Exploitation likelihood is *Very Likely* with *High* impact.

`path-traversal@postgres@volumen-postgres@postgres>escritura-volumen`

Unchecked

SQL/NoSQL-Injection: 1 / 1 Risk

Description (Tampering): [CWE 89](#)

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed or stored.

Impact

If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Detection Logic

Database accessed via typical database access protocols by in-scope clients.

Risk Rating

The risk rating depends on the sensitivity of the data stored inside the database.

False Positives

Database accesses by queries not consisting of parts controllable by the caller can be considered as false positives after individual review.

Mitigation (Development): SQL/NoSQL-Injection Prevention

Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V5 - Validation, Sanitization and Encoding Verification Requirements](#)

Cheat Sheet: [SQL_Injection_Prevention_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **SQL/NoSQL-Injection** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

SQL/NoSQL-Injection risk at **backend** against database **postgres** via **conexion_base_datos**: Exploitation likelihood is *Very Likely* with *High* impact.

`sql-nosql-injection@backend@postgres@backend>conexion-base-datos`

Unchecked

Cross-Site Scripting (XSS): 2 / 2 Risks

Description (Tampering): [CWE 79](#)

For each web application Cross-Site Scripting (XSS) risks might arise. In terms of the overall risk level take other applications running on the same domain into account as well.

Impact

If this risk remains unmitigated, attackers might be able to access individual victim sessions and steal or modify user data.

Detection Logic

In-scope web applications.

Risk Rating

The risk rating depends on the sensitivity of the data processed or stored in the web application.

False Positives

When the technical asset is not accessed via a browser-like component (i.e not by a human user initiating the request that gets passed through all components until it reaches the web application) this can be considered a false positive.

Mitigation (Development): XSS Prevention

Try to encode all values sent back to the browser and also handle DOM-manipulations in a safe way to avoid DOM-based XSS. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V5 - Validation, Sanitization and Encoding Verification Requirements](#)

Cheat Sheet: [Cross Site Scripting Prevention Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Cross-Site Scripting (XSS)** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Cross-Site Scripting (XSS) risk at **kubernetes-api-server**: Exploitation likelihood is *Likely* with *High* impact.

[cross-site-scripting@kubernetes-api-server](#)

Unchecked

Cross-Site Scripting (XSS) risk at **nginx**: Exploitation likelihood is *Likely* with *Medium* impact.

[cross-site-scripting@nginx](#)

Unchecked

Missing Authentication: 1 / 1 Risk

Description (Elevation of Privilege): [CWE 306](#)

Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes or stores sensitive data.

Impact

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, service-registry, waf, ids, and ips and in-process calls) should authenticate incoming requests when the asset processes or stores sensitive data. This is especially the case for all multi-tenant assets (there even non-sensitive ones).

Risk Rating

The risk rating (medium or high) depends on the sensitivity of the data sent across the communication link. Monitoring callers are exempted from this risk.

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Architecture): Authentication of Incoming Requests

Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

ASVS Chapter: [V2 - Authentication Verification Requirements](#)

Cheat Sheet: [Authentication_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Authentication** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Authentication covering communication link **routing_backend** from **ingress-controller** to **backend**: Exploitation likelihood is *Likely* with *Medium* impact.

`missing-authentication@ingress-controller>routing-backend@ingress-controller@backend`

Unchecked

Missing Hardening: 3 / 3 Risks

Description (Tampering): [CWE 16](#)

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Impact

If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Detection Logic

In-scope technical assets with RAA values of 55 % or higher. Generally for high-value targets like datastores, application servers, identity providers and ERP systems this limit is reduced to 40 %

Risk Rating

The risk rating depends on the sensitivity of the data processed or stored in the technical asset.

False Positives

Usually no false positives.

Mitigation (Operations): System Hardening

Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

ASVS Chapter: [V14 - Configuration Verification Requirements](#)

Cheat Sheet: [Attack Surface Analysis Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Hardening** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **github-actions-pipeline**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@github-actions-pipeline](#)

Unchecked

Missing Hardening risk at **postgres**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@postgres](#)

Unchecked

Missing Hardening risk at **volumen-postgres**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@volumen-postgres](#)

Unchecked

Accidental Secret Leak: 1 / 1 Risk

Description (Information Disclosure): [CWE 200](#)

Sourcecode repositories (including their histories) as well as artifact registries can accidentally contain secrets like checked-in or packaged-in passwords, API tokens, certificates, crypto keys, etc.

Impact

If this risk is unmitigated, attackers which have access to affected sourcecode repositories or artifact registries might find secrets accidentally checked-in.

Detection Logic

In-scope sourcecode repositories and artifact registries.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

False Positives

Usually no false positives.

Mitigation (Operations): Build Pipeline Hardening

Establish measures preventing accidental check-in or package-in of secrets into sourcecode repositories and artifact registries. This starts by using good `.gitignore` and `.dockerignore` files, but does not stop there. See for example tools like *"git-secrets"* or *"Talisman"* to have check-in preventive measures for secrets. Consider also to regularly scan your repositories for secrets accidentally checked-in using scanning tools like *"gitleaks"* or *"gitrob"*.

ASVS Chapter: [V14 - Configuration Verification Requirements](#)

Cheat Sheet: [Attack_Surface_Analysis_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Accidental Secret Leak** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Accidental Secret Leak risk at **docker-registry**: Exploitation likelihood is *Unlikely with High impact*.

[accidental-secret-leak@docker-registry](#)

Unchecked

Code Backdooring: 2 / 2 Risks

Description (Tampering): [CWE 912](#)

For each build-pipeline component Code Backdooring risks might arise where attackers compromise the build-pipeline in order to let backdoored artifacts be shipped into production. Aside from direct code backdooring this includes backdooring of dependencies and even of more lower-level build infrastructure, like backdooring compilers (similar to what the XcodeGhost malware did) or dependencies.

Impact

If this risk remains unmitigated, attackers might be able to execute code on and completely takeover production environments.

Detection Logic

In-scope development relevant technical assets which are either accessed by out-of-scope unmanaged developer clients and/or are directly accessed by any kind of internet-located (non-VPN) component or are themselves directly located on the internet.

Risk Rating

The risk rating depends on the confidentiality and integrity rating of the code being handled and deployed as well as the placement/calling of this technical asset on/from the internet.

False Positives

When the build-pipeline and sourcecode-repo is not exposed to the internet and considered fully trusted (which implies that all accessing clients are also considered fully trusted in terms of their patch management and applied hardening, which must be equivalent to a managed developer client environment) this can be considered a false positive after individual review.

Mitigation (Operations): Build Pipeline Hardening

Reduce the attack surface of backdooring the build pipeline by not directly exposing the build pipeline components on the public internet and also not exposing it in front of unmanaged (out-of-scope) developer clients. Also consider the use of code signing to prevent code modifications.

ASVS Chapter: [V10 - Malicious Code Verification Requirements](#)

Cheat Sheet: [Vulnerable_Dependency_Management_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Code Backdooring** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Code Backdooring risk at **docker-registry**: Exploitation likelihood is *Unlikely* with *High* impact.

[code-backdooring@docker-registry](#)

Unchecked

Code Backdooring risk at **github-actions-pipeline**: Exploitation likelihood is *Unlikely* with *High* impact.

[code-backdooring@github-actions-pipeline](#)

Unchecked

Container Base Image Backdooring: 10 / 10 Risks

Description (Tampering): [CWE 912](#)

When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

See for example:

<https://techcrunch.com/2018/06/15/tainted-crypto-mining-containers-pulled-from-docker-hub/>

Impact

If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Detection Logic

In-scope technical assets running as containers.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets.

False Positives

Fully trusted (i.e. reviewed and cryptographically signed or similar) base images of containers can be considered as false positives after individual review.

Mitigation (Operations): Container Infrastructure Hardening

Apply hardening of all container infrastructures (see for example the *CIS-Benchmarks for Docker and Kubernetes* and the *Docker Bench for Security*). Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's *Distroless* base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.

ASVS Chapter: [V10 - Malicious Code Verification Requirements](#)

Cheat Sheet: [Docker_Security_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS/CSVS applied?

Risk Findings

The risk **Container Base Image Backdooring** was found **10 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **backend**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@backend](#)

Unchecked

Container Base Image Backdooring risk at **kubernetes-api-server**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@kubernetes-api-server](#)

Unchecked

Container Base Image Backdooring risk at **postgres**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@postgres](#)

Unchecked

Container Base Image Backdooring risk at **prometheus**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@prometheus](#)

Unchecked

Container Base Image Backdooring risk at **volumen-postgres**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@volumen-postgres](#)

Unchecked

Container Base Image Backdooring risk at **cadvisor**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[container-baseimage-backdooring@cadvisor](#)

Unchecked

Container Base Image Backdooring risk at **grafana**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[container-baseimage-backdooring@grafana](#)

Unchecked

Container Base Image Backdooring risk at **ingress-controller**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@ingress-controller

Unchecked

Container Base Image Backdooring risk at **nginx**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@nginx

Unchecked

Container Base Image Backdooring risk at **node-exporter**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@node-exporter

Unchecked

Cross-Site Request Forgery (CSRF): 2 / 2 Risks

Description (Spoofing): [CWE 352](#)

When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.

Impact

If this risk remains unmitigated, attackers might be able to trick logged-in victim users into unwanted actions within the web application by visiting an attacker controlled web site.

Detection Logic

In-scope web applications accessed via typical web access protocols.

Risk Rating

The risk rating depends on the integrity rating of the data sent across the communication link.

False Positives

Web applications passing the authentication state via custom headers instead of cookies can eventually be false positives. Also when the web application is not accessed via a browser-like component (i.e not by a human user initiating the request that gets passed through all components until it reaches the web application) this can be considered a false positive.

Mitigation (Development): CSRF Prevention

Try to use anti-CSRF tokens or the double-submit patterns (at least for logged-in requests). When your authentication scheme depends on cookies (like session or token cookies), consider marking them with the same-site flag. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V4 - Access Control Verification Requirements](#)

Cheat Sheet: [Cross-Site_Request_Forgery_Prevention_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Cross-Site Request Forgery (CSRF)** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Cross-Site Request Forgery (CSRF) risk at **nginx** via **routing_frontend** from **ingress-controller**: Exploitation likelihood is *Very Likely* with *Low* impact.

[cross-site-request-forgery@nginx@ingress-controller>routing-frontend](#)

Unchecked

Cross-Site Request Forgery (CSRF) risk at **kubernetes-api-server** via **service_discovery_k8s_api** from **prometheus**: Exploitation likelihood is *Likely* with *Low* impact.

[cross-site-request-forgery@kubernetes-api-server@prometheus>service-discovery-k8s-api](#)

Unchecked

Missing Build Infrastructure: 1 / 1 Risk

Description (Tampering): [CWE 1127](#)

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Impact

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Detection Logic

Models with in-scope custom-developed parts missing in-scope development (code creation) and build infrastructure components (devops-client, sourcecode-repo, build-pipeline, etc.).

Risk Rating

The risk rating depends on the highest sensitivity of the in-scope assets running custom-developed parts.

False Positives

Models not having any custom-developed parts can be considered as false positives after individual review.

Mitigation (Architecture): Build Pipeline Hardening

Include the build infrastructure in the model.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Attack_Surface_Analysis_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Build Infrastructure** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Build Infrastructure in the threat model (referencing asset **backend** as an example):
Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-build-infrastructure@backend](#)

Unchecked

Missing Vault (Secret Storage): 1 / 1 Risk

Description (Information Disclosure): [CWE 522](#)

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Impact

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Detection Logic

Models without a Vault (Secret Storage).

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

False Positives

Models where no technical assets have any kind of sensitive config data to protect can be considered as false positives after individual review.

Mitigation (Architecture): Vault (Secret Storage)

Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

ASVS Chapter: [V6 - Stored Cryptography Verification Requirements](#)

Cheat Sheet: [Cryptographic_Storage_Cheat_Sheet](#)

Check

Is a Vault (Secret Storage) in place?

Risk Findings

The risk **Missing Vault (Secret Storage)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Vault (Secret Storage) in the threat model (referencing asset **kubernetes-api-server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-vault@kubernetes-api-server](#)

Accepted

2026-09-06

Equipo DevOps

TP14-04

Kubernetes monta el token efímero de prometheus-sa. El ClusterRole sólo permite get/list/watch de pods, services, endpoints y nodes, no Secrets. El laboratorio no incorpora un vault externo.

Server-Side Request Forgery (SSRF): 6 / 6 Risks

Description (Information Disclosure): [CWE 918](#)

When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Impact

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Detection Logic

In-scope non-client systems accessing (using outgoing communication links) targets with either HTTP or HTTPS protocol.

Risk Rating

The risk rating (low or medium) depends on the sensitivity of the data assets receivable via web protocols from targets within the same network trust-boundary as well on the sensitivity of the data assets receivable via web protocols from the target asset itself. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF.

False Positives

Servers not sending outgoing web requests can be considered as false positives after review.

Mitigation (Development): SSRF Prevention

Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: [V12 - File and Resources Verification Requirements](#)

Cheat Sheet: [Server_Side_Request_Forgery_Prevention_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Server-Side Request Forgery (SSRF)** was found **6 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Server-Side Request Forgery (SSRF) risk at **github-actions-pipeline** server-side web-requesting the target **docker-registry** via **publica-backend-historico**: Exploitation likelihood is *Unlikely* with *Medium* impact.

`server-side-request-forgery@github-actions-pipeline@docker-registry@github-actions-pipeline>publica-backend-historico`

Unchecked

Server-Side Request Forgery (SSRF) risk at **grafana** server-side web-requesting the target **prometheus** via **consulta_datos_prometheus**: Exploitation likelihood is *Unlikely* with *Medium* impact.

`server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus`

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **backend** via **scrape_backend**: Exploitation likelihood is *Unlikely* with *Medium* impact.

`server-side-request-forgery@prometheus@backend@prometheus>scrape-backend`

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **cadvisor** via **scrape_cadvisor**: Exploitation likelihood is *Unlikely* with *Medium* impact.

`server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor`

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **kubernetes-api-server** via **service_discovery_k8s_api**: Exploitation likelihood is *Unlikely* with *Medium* impact.

`server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api`

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **node-exporter** via **scrape_node_exporter**: Exploitation likelihood is *Unlikely* with *Medium* impact.

`server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter`

Unchecked

Unchecked Deployment: 2 / 2 Risks

Description (Tampering): [CWE 1127](#)

For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Impact

If this risk remains unmitigated, vulnerabilities in custom-developed software or their dependencies might not be identified during continuous deployment cycles.

Detection Logic

All development-relevant technical assets.

Risk Rating

The risk rating depends on the highest rating of the technical assets and data assets processed by deployment-receiving targets.

False Positives

When the build-pipeline does not build any software components it can be considered a false positive after individual review.

Mitigation (Architecture): Build Pipeline Hardening

Apply DevSecOps best-practices and use scanning tools to identify vulnerabilities in source- or byte-code, dependencies, container layers, and optionally also via dynamic scans against running test systems.

ASVS Chapter: [V14 - Configuration Verification Requirements](#)

Cheat Sheet: [Vulnerable Dependency Management Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unchecked Deployment** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unchecked Deployment risk at **github-actions-pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[unchecked-deployment@github-actions-pipeline](#)

Unchecked

Low Risk Severity

Unchecked Deployment risk at **docker-registry**: Exploitation likelihood is *Unlikely* with *Low* impact.

[unchecked-deployment@docker-registry](#)

Accepted 2026-09-06 Equipo DevOps TP14-03

Los Secrets DOCKERHUB_USERNAME y DOCKERHUB_TOKEN controlan el acceso al registro. El workflow publica el backend histórico desde main y develop sin aprobación manual, por lo que el riesgo residual queda aceptado.

Unencrypted Communication: 1 / 1 Risk

Description (Information Disclosure): [CWE 319](#)

Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Impact

If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Detection Logic

Unencrypted technical communication links of in-scope technical assets (excluding monitoring traffic as well as local-file-access and in-process-library-call) transferring sensitive data.

Risk Rating

Depending on the confidentiality rating of the transferred data-assets either medium or high risk.

False Positives

When all sensitive data sent over the communication link is already fully encrypted on document or data level. Also intra-container/pod communication can be considered false positive when container orchestration platform handles encryption.

Mitigation (Operations): Encryption of Communication Links

Apply transport layer encryption to the communication link.

ASVS Chapter: [V9 - Communication Verification Requirements](#)

Cheat Sheet: [Transport_Layer_Protection_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Communication** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unencrypted Communication named **conexion_base_datos** between **backend** and **postgres** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-communication@backend>conexion-base-datos@backend@postgres](#)

Accepted 2026-09-06 Equipo DevOps TP14-02

La conexión psycopg2 usa el protocolo PostgreSQL sin TLS. Se acepta en este laboratorio local porque backend y PostgreSQL son ClusterIP dentro del clúster k3d y no se publica el puerto 5432. El riesgo residual se acepta porque el laboratorio no utiliza NetworkPolicy.

Unencrypted Technical Assets: 5 / 5 Risks

Description (Information Disclosure): [CWE 311](#)

Due to the confidentiality rating of the technical asset itself and/or the processed data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Impact

If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Detection Logic

In-scope unencrypted technical assets (excluding reverse-proxy, load-balancer, waf, ids, ips and embedded components like library) storing data assets rated at least as confidential or critical. For technical assets storing data assets rated as strictly-confidential or mission-critical the encryption must be of type data-with-enduser-individual-key.

Risk Rating

Depending on the confidentiality rating of the stored data-assets either medium or high risk.

False Positives

When all sensitive data stored within the asset is already fully encrypted on document or data level.

Mitigation (Operations): Encryption of Technical Asset

Apply encryption to the technical asset.

ASVS Chapter: [V6 - Stored Cryptography Verification Requirements](#)

Cheat Sheet: [Cryptographic Storage Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Technical Assets** was found **5 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unencrypted Technical Asset named **backend**: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@backend](#)

Unchecked

Unencrypted Technical Asset named **postgres** missing enduser-individual encryption with data-with-enduser-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@postgres](#)

Unchecked

Unencrypted Technical Asset named **prometheus**: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@prometheus](#)

Unchecked

Unencrypted Technical Asset named **volumen-postgres** missing enduser-individual encryption with data-with-enduser-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@volumen-postgres](#)

Unchecked

Unencrypted Technical Asset named **grafana**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[unencrypted-asset@grafana](#)

Unchecked

Unguarded Access From Internet: 1 / 1 Risk

Description (Elevation of Privilege): [CWE 501](#)

Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Impact

If this risk is unmitigated, attackers might be able to directly attack sensitive systems without any hardening components in-between due to them being directly exposed on the internet.

Detection Logic

In-scope technical assets (excluding load-balancer) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) when accessed directly from the internet. All web-server, web-application, reverse-proxy, waf, and gateway assets are exempted from this risk when they do not consist of custom developed code and the data-flow only consists of HTTP or FTP protocols. Access from monitoring systems as well as VPN-protected connections are exempted.

Risk Rating

The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

False Positives

When other means of filtering client requests are applied equivalent of reverse-proxy, waf, or gateway components.

Mitigation (Architecture): Encapsulation of Technical Asset

Encapsulate the asset behind a guarding service, application, or reverse-proxy. For admin maintenance a bastion-host should be used as a jump-server. For file transfer a store-and-forward-host should be used as an indirect file exchange platform.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Attack_Surface_Analysis_Cheat_Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unguarded Access From Internet** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unguarded Access from Internet of backend by ingress-controller via routing_backend:
Exploitation likelihood is *Very Likely* with *Low* impact.

`unguarded-access-from-internet@backend@ingress-controller@ingress-controller>routing-backend`

Unchecked

Missing Network Segmentation: 2 / 2 Risks

Description (Elevation of Privilege): [CWE 1008](#)

Highly sensitive assets and/or datastores residing in the same network segment than other lower sensitive assets (like webserver or content management systems etc.) should be better protected by a network segmentation trust-boundary.

Impact

If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are not separated by network segmentation.

Detection Logic

In-scope technical assets with high sensitivity and RAA values as well as datastores when surrounded by assets (without a network trust-boundary in-between) which are of type client-system, web-server, web-application, cms, web-service-rest, web-service-soap, build-pipeline, sourcecode-repository, monitoring, or similar and there is no direct connection between these (hence no requirement to be so close to each other).

Risk Rating

Default is low risk. The risk is increased to medium when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

False Positives

When all assets within the network segmentation trust-boundary are hardened and protected to the same extend as if all were containing/processing highly sensitive data.

Mitigation (Operations): Network Segmentation

Apply a network segmentation trust-boundary around the highly sensitive assets and/or datastores.

ASVS Chapter: [V1 - Architecture, Design and Threat Modeling Requirements](#)

Cheat Sheet: [Attack Surface Analysis Cheat Sheet](#)

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Network Segmentation** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Low Risk Severity

Missing Network Segmentation to further encapsulate and protect **github-actions-pipeline** against unrelated lower protected assets in the same network segment, which might be easier to compromise by attackers: Exploitation likelihood is *Unlikely* with *Low* impact.

[missing-network-segmentation@github-actions-pipeline](#)

Unchecked

Missing Network Segmentation to further encapsulate and protect **postgres** against unrelated lower protected assets in the same network segment, which might be easier to compromise by attackers: Exploitation likelihood is *Unlikely* with *Low* impact.

[missing-network-segmentation@postgres](#)

Unchecked

Identified Risks by Technical Asset

In total **42 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 2 as high, 6 as elevated, 31 as medium, and 3 as low.**

These risks are distributed across **13 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

backend: 7 / 7 Risks

Description

Servidor de aplicación que gestiona la lógica de las notas y expone métricas internas para Prometheus.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

SQL/NoSQL-Injection risk at **backend** against database **postgres** via **conexion_base_datos**: Exploitation likelihood is *Very Likely* with *High* impact.

[sql-nosql-injection@backend@postgres@backend>conexion-base-datos](#)

Unchecked

Elevated Risk Severity

Missing Authentication covering communication link **routing_backend** from **ingress-controller** to **backend**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-authentication@ingress-controller>routing-backend@ingress-controller@backend](#)

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **backend**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@backend](#)

Unchecked

Unencrypted Technical Asset named **backend**: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@backend](#)

Unchecked

Missing Build Infrastructure in the threat model (referencing asset **backend** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-build-infrastructure@backend](#)

Unchecked

Unguarded Access from Internet of backend by ingress-controller via routing_backend:
Exploitation likelihood is *Very Likely* with *Low* impact.

unguarded-access-from-internet@backend@ingress-controller@ingress-controller>routing-backend

Unchecked

Unencrypted Communication named conexion_base_datos between backend and postgres transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-communication@backend>conexion-base-datos@backend@postgres

Accepted 2026-09-06 Equipo DevOps TP14-02

La conexión psycopg2 usa el protocolo PostgreSQL sin TLS. Se acepta en este laboratorio local porque backend y PostgreSQL son ClusterIP dentro del clúster k3d y no se publica el puerto 5432. El riesgo residual se acepta porque el laboratorio no utiliza NetworkPolicy.

Asset Information

ID:	backend
Type:	process
Usage:	business
RAA:	34 %
Size:	component
Technology:	web-service-rest
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	contenido-notas, credenciales-bd
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	Equipo DevOps
Confidentiality:	confidential (rated 4 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	important (rated 3 in scale of 5)
CIA-Justification:	Procesa las credenciales de base de datos y de usuario, y toda la lógica de negocio de las notas.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

conexion_base_datos (outgoing)

Conexión de red activa iniciada por la app de Flask para realizar consultas de inserción y lectura en PostgreSQL.

Target:	postgres
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	contenido-notas, credenciales-bd
Data Received:	none

Incoming Communication Links: 2

Source technical asset names are clickable and link to the corresponding chapter.

scrape_backend (incoming)

Peticiones de scraping HTTP GET periódicas hacia el endpoint /metrics del Backend de Flask (Service backend-service); las métricas viajan en la respuesta, no en el pedido.

Source:	prometheus
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	none
Data Sent:	metricas-monitoreo

routing_backend (incoming)

Reenvía el tráfico de los paths `"/api"` y `"/health"` directo al Service `backend-service`, sin pasar por `nginx`.

Source:	ingress-controller
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	contenido-notas
Data Sent:	none

postgres: 5 / 5 Risks

Description

Base de datos relacional PostgreSQL encargada de resguardar los datos de las notas de forma persistente.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

Path-Traversal risk at **postgres** against filesystem **volumen-postgres** via **escritura_volumen**: Exploitation likelihood is *Very Likely* with *High* impact.

[path-traversal@postgres@volumen-postgres@postgres>escritura-volumen](#)

Unchecked

Elevated Risk Severity

Missing Hardening risk at **postgres**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@postgres](#)

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **postgres**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@postgres](#)

Unchecked

Unencrypted Technical Asset named **postgres** missing enduser-individual encryption with **data-with-enduser-individual-key**: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@postgres](#)

Unchecked

Low Risk Severity

Missing Network Segmentation to further encapsulate and protect **postgres** against unrelated lower protected assets in the same network segment, which might be easier to compromise by attackers: Exploitation likelihood is *Unlikely* with *Low* impact.

[missing-network-segmentation@postgres](#)

Unchecked

Asset Information

ID:	postgres
Type:	datastore
Usage:	business
RAA:	100 %
Size:	component
Technology:	database
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	contenido-notas, credenciales-bd
Data Stored:	contenido-notas, credenciales-bd
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Almacena de forma persistente todas las notas y depende de ella toda la aplicación.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

escritura_volumen (outgoing)

PostgreSQL escribe sus archivos de datos en el PersistentVolumeClaim montado (almacenamiento local del clúster k3d).

Target:	volumen-postgres
Protocol:	local-file-access

Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	contenido-notas, credenciales-bd
Data Received:	none

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

conexion_base_datos (incoming)

Conexión de red activa iniciada por la app de Flask para realizar consultas de inserción y lectura en PostgreSQL.

Source:	backend
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	contenido-notas, credenciales-bd
Data Sent:	none

github-actions-pipeline: 5 / 5 Risks

Description

Runner externo que construye y publica en Docker Hub la imagen del backend histórico desde main o develop.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **github-actions-pipeline**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@github-actions-pipeline](#)

Unchecked

Medium Risk Severity

Code Backdooring risk at **github-actions-pipeline**: Exploitation likelihood is *Unlikely* with *High* impact.

[code-backdooring@github-actions-pipeline](#)

Unchecked

Server-Side Request Forgery (SSRF) risk at **github-actions-pipeline** server-side web-requesting the target **docker-registry** via **publica-backend-historico**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[server-side-request-forgery@github-actions-pipeline@docker-registry@github-actions-pipeline>publica-backend-historico](#)

Unchecked

Unchecked Deployment risk at **github-actions-pipeline**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[unchecked-deployment@github-actions-pipeline](#)

Unchecked

Low Risk Severity

Missing Network Segmentation to further encapsulate and protect **github-actions-pipeline** against unrelated lower protected assets in the same network segment, which might be easier to compromise by attackers: Exploitation likelihood is *Unlikely* with *Low* impact.

[missing-network-segmentation@github-actions-pipeline](#)

Unchecked

Asset Information

ID:	github-actions-pipeline
Type:	process
Usage:	devops
RAA:	69 %
Size:	service
Technology:	build-pipeline
Tags:	none
Internet:	true
Machine:	serverless
Encryption:	transparent
Multi-Tenant:	true
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	credenciales-registro
Data Stored:	none
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps / GitHub	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Controla la publicación de imágenes y se ejecuta fuera del clúster local.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

publica-backend-historico (outgoing)

Publicación HTTPS autenticada del backend histórico desde main o develop; no publica la app integrada.

Target:	docker-registry
Protocol:	https
Encrypted:	true

Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Sent: credenciales-registro
Data Received: none

kubernetes-api-server: 4 / 4 Risks

Description

Plano de control del clúster. Prometheus lo consulta para el service discovery dinámico (kubernetes_sd_configs) en vez de usar hostnames fijos, autenticado con el token de la ServiceAccount "prometheus-sa" y autorizado por un ClusterRole de solo lectura (pods, services, endpoints, nodes).

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Cross-Site Scripting (XSS) risk at **kubernetes-api-server**: Exploitation likelihood is *Likely* with *High* impact.

[cross-site-scripting@kubernetes-api-server](#)

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **kubernetes-api-server**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@kubernetes-api-server](#)

Unchecked

Cross-Site Request Forgery (CSRF) risk at **kubernetes-api-server** via **service_discovery_k8s_api** from **prometheus**: Exploitation likelihood is *Likely* with *Low* impact.

[cross-site-request-forgery@kubernetes-api-server@prometheus>service-discovery-k8s-api](#)

Unchecked

Missing Vault (Secret Storage) in the threat model (referencing asset **kubernetes-api-server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

[missing-vault@kubernetes-api-server](#)

Accepted

2026-09-06 Equipo DevOps

TP14-04

Kubernetes monta el token efímero de prometheus-sa. El ClusterRole sólo permite get/list/watch de pods, services, endpoints y nodes, no Secrets. El laboratorio no incorpora un vault externo.

Asset Information

ID:	kubernetes-api-server
Type:	process
Usage:	devops

RAA:	16 %
Size:	system
Technology:	application-server
Tags:	none
Internet:	false
Machine:	container
Encryption:	transparent
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	metadata-descubrimiento-k8s
Data Stored:	none
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	mission-critical (rated 5 in scale of 5)
Availability:	mission-critical (rated 5 in scale of 5)
CIA-Justification:	Es el plano de control de todo el clúster; comprometerlo compromete todo lo demás.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

service_discovery_k8s_api (incoming)

Prometheus consulta al API Server con el token de "prometheus-sa" para descubrir Pods con label app=backend y los Endpoints de node-exporter/cadvisor (kubernetes_sd_configs, roles pod y endpoints).

Source:	prometheus
Protocol:	https
Encrypted:	true
Authentication:	token
Authorization:	technical-user
Read-Only:	true

Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Received: token-service-account-prometheus
Data Sent: metadata-descubrimiento-k8s

nginx: 3 / 3 Risks

Description

Pod Nginx que sirve el frontend estático de la Notes App detrás del Ingress.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Cross-Site Scripting (XSS) risk at **nginx**: Exploitation likelihood is *Likely* with *Medium* impact.

[cross-site-scripting@nginx](#)

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **nginx**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[container-baseimage-backdooring@nginx](#)

Unchecked

Cross-Site Request Forgery (CSRF) risk at **nginx** via **routing_frontend** from **ingress-controller**: Exploitation likelihood is *Very Likely* with *Low* impact.

[cross-site-request-forgery@nginx@ingress-controller>routing-frontend](#)

Unchecked

Asset Information

ID:	nginx
Type:	process
Usage:	business
RAA:	9 %
Size:	component
Technology:	web-server
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false

Client by Human: false
 Data Processed: contenido-notas
 Data Stored: none
 Formats Accepted: none of the special data formats accepted

Asset Rating

Owner: Equipo DevOps
 Confidentiality: internal (rated 2 in scale of 5)
 Integrity: important (rated 3 in scale of 5)
 Availability: important (rated 3 in scale of 5)
 CIA-Justification: Sirve la interfaz al usuario; sólo recibe tráfico ya filtrado por el Ingress.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

[routing_frontend \(incoming\)](#)

Reenvía el tráfico del path "/" al Service frontend-service dentro del clúster.

Source: ingress-controller
 Protocol: http
 Encrypted: false
 Authentication: none
 Authorization: none
 Read-Only: false
 Usage: business
 Tags: none
 VPN: false
 IP-Filtered: false
 Data Received: contenido-notas
 Data Sent: none

volumen-postgres: 3 / 3 Risks

Description

PersistentVolumeClaim de Kubernetes donde PostgreSQL guarda físicamente los archivos de la base de datos.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **volumen-postgres**: Exploitation likelihood is *Likely* with *Medium* impact.

[missing-hardening@volumen-postgres](#)

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **volumen-postgres**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@volumen-postgres](#)

Unchecked

Unencrypted Technical Asset named **volumen-postgres** missing enduser-individual encryption with data-with-enduser-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@volumen-postgres](#)

Unchecked

Asset Information

ID:	volumen-postgres
Type:	datastore
Usage:	business
RAA:	49 %
Size:	component
Technology:	local-file-system
Tags:	none
Internet:	false
Machine:	container
Encryption:	none

Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	none
Data Stored:	contenido-notas, credenciales-bd
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps
Confidentiality:	confidential (rated 4 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	important (rated 3 in scale of 5)
CIA-Justification:	Es el almacenamiento físico final de todos los datos de la base; su pérdida es irreversible.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

escritura_volumen (incoming)

PostgreSQL escribe sus archivos de datos en el PersistentVolumeClaim montado (almacenamiento local del clúster k3d).

Source:	postgres
Protocol:	local-file-access
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	contenido-notas, credenciales-bd
Data Sent:	none

cadvisor: 1 / 1 Risk

Description

DaemonSet encargado de exponer de forma pasiva las métricas de rendimiento internas de los contenedores del clúster.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **cadvisor**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[container-baseimage-backdooring@cadvisor](#)

Unchecked

Asset Information

ID:	cadvisor
Type:	process
Usage:	devops
RAA:	1 %
Size:	component
Technology:	monitoring
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	metricas-monitoreo
Data Stored:	none
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps
--------	---------------

Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Expone métricas de solo lectura de contenedores, sin datos de negocio.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

scrape_cadvisor (incoming)

Peticiones HTTP periódicas al Service cAdvisor en el puerto 8080 para monitorear los contenedores del clúster; las métricas viajan en la respuesta, no en el pedido.

Source:	prometheus
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	none
Data Sent:	metricas-monitoreo

docker-registry: 3 / 3 Risks

Description

Registro público donde el pipeline de CI/CD publica la imagen del backend.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Accidental Secret Leak risk at **docker-registry**: Exploitation likelihood is *Unlikely* with *High* impact.

[accidental-secret-leak@docker-registry](#)

Unchecked

Code Backdooring risk at **docker-registry**: Exploitation likelihood is *Unlikely* with *High* impact.

[code-backdooring@docker-registry](#)

Unchecked

Low Risk Severity

Unchecked Deployment risk at **docker-registry**: Exploitation likelihood is *Unlikely* with *Low* impact.

[unchecked-deployment@docker-registry](#)

Accepted 2026-09-06 Equipo DevOps TP14-03

Los Secrets DOCKERHUB_USERNAME y DOCKERHUB_TOKEN controlan el acceso al registro. El workflow publica el backend histórico desde main y develop sin aprobación manual, por lo que el riesgo residual queda aceptado.

Asset Information

ID:	docker-registry
Type:	process
Usage:	devops
RAA:	44 %
Size:	component
Technology:	artifact-registry
Tags:	none
Internet:	true
Machine:	serverless
Encryption:	transparent

Multi-Tenant: true
Redundant: false
Custom-Developed: false
Client by Human: false
Data Processed: credenciales-registro
Data Stored: none
Formats Accepted: none of the special data formats accepted

Asset Rating

Owner: Docker Hub (terceros)
Confidentiality: internal (rated 2 in scale of 5)
Integrity: important (rated 3 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Un push accidental o no controlado a un registro público expondría las imágenes de la aplicación.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

publica-backend-historico (incoming)

Publicación HTTPS autenticada del backend histórico desde main o develop; no publica la app integrada.

Source: github-actions-pipeline
Protocol: https
Encrypted: true
Authentication: credentials
Authorization: technical-user
Read-Only: false
Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Received: credenciales-registro
Data Sent: none

grafana: 3 / 3 Risks

Description

Plataforma web interactiva para visualizar dashboards basados en las métricas consolidadas.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **grafana**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@grafana

Unchecked

Server-Side Request Forgery (SSRF) risk at **grafana** server-side web-requesting the target **prometheus** via **consulta_datos_prometheus**: Exploitation likelihood is *Unlikely* with *Medium* impact.

server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus

Unchecked

Unencrypted Technical Asset named **grafana**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@grafana

Unchecked

Asset Information

ID:	grafana
Type:	process
Usage:	devops
RAA:	2 %
Size:	component
Technology:	monitoring
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false

Client by Human: false
Data Processed: credenciales-usuario, metricas-monitoreo
Data Stored: none
Formats Accepted: none of the special data formats accepted

Asset Rating

Owner: Equipo DevOps
Confidentiality: internal (rated 2 in scale of 5)
Integrity: operational (rated 2 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Solo visualiza métricas ya consolidadas por Prometheus.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

consulta_datos_prometheus (outgoing)

Peticiones HTTP periódicas dirigidas al Service prometheus en el puerto 9090 para obtener la información de los gráficos en tiempo real; las métricas viajan en la respuesta, no en el pedido.

Target: prometheus
Protocol: http
Encrypted: false
Authentication: none
Authorization: none
Read-Only: true
Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Sent: none
Data Received: metricas-monitoreo

ingress-controller: 1 / 1 Risk

Description

Ingress de Kubernetes (ingress-nginx) que termina TLS y enruta por path hacia los Services internos - "/" al frontend, "/api" y "/health" directo al backend - según el chart Helm de TP10.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **ingress-controller**: Exploitation likelihood is *Unlikely with Medium impact*.

[container-baseimage-backdooring@ingress-controller](#)

Unchecked

Asset Information

ID:	ingress-controller
Type:	process
Usage:	business
RAA:	3 %
Size:	component
Technology:	load-balancer
Tags:	none
Internet:	true
Machine:	container
Encryption:	transparent
Multi-Tenant:	false
Redundant:	true
Custom-Developed:	false
Client by Human:	false
Data Processed:	contenido-notas
Data Stored:	none
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps
--------	---------------

Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	important	(rated 3 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Es la única puerta de entrada pública hacia el clúster; termina TLS antes de reenviar el tráfico a los Pods internos.	

Outgoing Communication Links: 2

Target technical asset names are clickable and link to the corresponding chapter.

`routing_frontend` (outgoing)

Reenvía el tráfico del path "/" al Service frontend-service dentro del clúster.

Target:	<code>nginx</code>
Protocol:	<code>http</code>
Encrypted:	<code>false</code>
Authentication:	<code>none</code>
Authorization:	<code>none</code>
Read-Only:	<code>false</code>
Usage:	<code>business</code>
Tags:	<code>none</code>
VPN:	<code>false</code>
IP-Filtered:	<code>false</code>
Data Sent:	<code>contenido-notas</code>
Data Received:	<code>none</code>

`routing_backend` (outgoing)

Reenvía el tráfico de los paths `/api` y `/health` directo al Service backend-service, sin pasar por `nginx`.

Target:	<code>backend</code>
Protocol:	<code>http</code>
Encrypted:	<code>false</code>
Authentication:	<code>none</code>
Authorization:	<code>none</code>
Read-Only:	<code>false</code>
Usage:	<code>business</code>
Tags:	<code>none</code>
VPN:	<code>false</code>
IP-Filtered:	<code>false</code>

Data Sent: contenido-notas
Data Received: none

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

`peticiones_usuario` (incoming)

Tráfico HTTPS hacia el Ingress para usar la interfaz y las operaciones de consulta, creación y eliminación de notas.

Source: cliente-web
Protocol: https
Encrypted: true
Authentication: none
Authorization: none
Read-Only: false
Usage: business
Tags: none
VPN: false
IP-Filtered: false
Data Received: contenido-notas
Data Sent: none

node-exporter: 1 / 1 Risk

Description

DaemonSet que expone de forma pasiva las métricas del hardware de cada nodo del clúster.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **node-exporter**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[container-baseimage-backdooring@node-exporter](#)

Unchecked

Asset Information

ID:	node-exporter
Type:	process
Usage:	devops
RAA:	1 %
Size:	component
Technology:	monitoring
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	metricas-monitoreo
Data Stored:	none
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps	
Confidentiality:	internal	(rated 2 in scale of 5)

Integrity: operational (rated 2 in scale of 5)
Availability: operational (rated 2 in scale of 5)
CIA-Justification: Expone métricas de solo lectura del host, sin datos de negocio.

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

scrape_node_exporter (incoming)

Peticiones HTTP periódicas hacia el Service node-exporter en el puerto 9100; las métricas viajan en la respuesta, no en el pedido.

Source:	prometheus
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	none
Data Sent:	metricas-monitoreo

prometheus: 6 / 6 Risks

Description

Sistema de monitoreo que recopila (scrapes) las métricas de rendimiento del ecosistema de la Notes App, y descubre sus targets (backend, node-exporter, cadvisor) de forma dinámica consultando al API Server de Kubernetes por label, en vez de usar hostnames fijos.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **prometheus**: Exploitation likelihood is *Unlikely* with *High* impact.

[container-baseimage-backdooring@prometheus](#)

Unchecked

Unencrypted Technical Asset named **prometheus**: Exploitation likelihood is *Unlikely* with *High* impact.

[unencrypted-asset@prometheus](#)

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **backend** via **scrape_backend**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[server-side-request-forgery@prometheus@backend@prometheus>scrape-backend](#)

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **cadvisor** via **scrape_cadvisor**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor](#)

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **kubernetes-api-server** via **service_discovery_k8s_api**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api](#)

Unchecked

Server-Side Request Forgery (SSRF) risk at **prometheus** server-side web-requesting the target **node-exporter** via **scrape_node_exporter**: Exploitation likelihood is *Unlikely* with *Medium* impact.

[server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter](#)

Unchecked

Asset Information

ID:	prometheus
Type:	process
Usage:	devops
RAA:	8 %
Size:	component
Technology:	monitoring
Tags:	none
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	metadata-descubrimiento-k8s, metricas-monitoreo, token-service-account-prometheus
Data Stored:	metricas-monitoreo
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Equipo DevOps	
Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Consolida métricas operativas; no almacena datos de negocio de los usuarios.	

Outgoing Communication Links: 4

Target technical asset names are clickable and link to the corresponding chapter.

[service_discovery_k8s_api](#) (outgoing)

Prometheus consulta al API Server con el token de "prometheus-sa" para descubrir Pods con label app=backend y los Endpoints de node-exporter/cadvisor (kubernetes_sd_configs, roles pod y endpoints).

Target:	kubernetes-api-server
Protocol:	https
Encrypted:	true
Authentication:	token
Authorization:	technical-user
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	token-service-account-prometheus
Data Received:	metadata-descubrimiento-k8s

scrape_node_exporter (outgoing)

Peticiones HTTP periódicas hacia el Service node-exporter en el puerto 9100; las métricas viajan en la respuesta, no en el pedido.

Target:	node-exporter
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	true
Usage:	devops
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	none
Data Received:	metricas-monitoreo

scrape_cadvisor (outgoing)

Peticiones HTTP periódicas al Service cAdvisor en el puerto 8080 para monitorear los contenedores del clúster; las métricas viajan en la respuesta, no en el pedido.

Target:	cadvisor
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	true

Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Sent: none
Data Received: metricas-monitoreo

scrape_backend (outgoing)

Peticiones de scraping HTTP GET periódicas hacia el endpoint /metrics del Backend de Flask (Service backend-service); las métricas viajan en la respuesta, no en el pedido.

Target: backend
Protocol: http
Encrypted: false
Authentication: none
Authorization: none
Read-Only: true
Usage: devops
Tags: none
VPN: false
IP-Filtered: false
Data Sent: none
Data Received: metricas-monitoreo

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

consulta_datos_prometheus (incoming)

Peticiones HTTP periódicas dirigidas al Service prometheus en el puerto 9090 para obtener la información de los gráficos en tiempo real; las métricas viajan en la respuesta, no en el pedido.

Source: grafana
Protocol: http
Encrypted: false
Authentication: none
Authorization: none
Read-Only: true
Usage: devops
Tags: none

VPN: false
IP-Filtered: false
Data Received: none
Data Sent: metricas-monitoreo

cliente-web: 0 / 0 Risks

Description

El cliente HTTP (navegador) en el dispositivo del usuario que inicia el acceso a la Notes App a través de internet de forma pública.

Identified Risks of Asset

No risks were identified.

Asset Information

ID:	cliente-web
Type:	external-entity
Usage:	business
RAA:	11 %
Size:	component
Technology:	browser
Tags:	none
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	true
Data Processed:	contenido-notas
Data Stored:	none
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	Usuario externo
Confidentiality:	internal (rated 2 in scale of 5)
Integrity:	operational (rated 2 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Es un dispositivo fuera del control del equipo DevOps.

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

peticiones_usuario (outgoing)

Tráfico HTTPS hacia el Ingress para usar la interfaz y las operaciones de consulta, creación y eliminación de notas.

Target:	ingress-controller
Protocol:	https
Encrypted:	true
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	contenido-notas
Data Received:	none

Identified Data Breach Probabilities by Data Asset

In total **42 potential risks** have been identified during the threat modeling process of which **0 are rated as critical, 2 as high, 6 as elevated, 31 as medium, and 3 as low.**

These risks are distributed across **7 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

contenido-notas: 23 / 23 Risks

Título y contenido de las notas creadas por los usuarios de la aplicación.

ID:	contenido-notas	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:		
Owner:		
Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	important	(rated 3 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Es el dato de negocio principal de la aplicación; su pérdida afecta a los usuarios.	
Processed by:	backend, cliente-web, ingress-controller, nginx, postgres	
Stored by:	postgres, volumen-postgres	
Sent via:	routing_frontend, routing_backend, peticiones_usuario, escritura_volumen, conexion_base_datos	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 23 remaining risks:	

Probable: container-baseimage-backdooring@backend

Probable: container-baseimage-backdooring@ingress-controller

Probable: container-baseimage-backdooring@nginx

Probable: container-baseimage-backdooring@postgres

Probable: container-baseimage-backdooring@volumen-postgres

Probable: path-traversal@postgres@volumen-postgres@postgres>escritura-volumen

Probable: sql-nosql-injection@backend@postgres@backend>conexion-base-datos

Possible: cross-site-scripting@nginx

Possible: missing-authentication@ingress-controller>routing-backend@ingress-controller@backend

Possible: server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus

Possible: server-side-request-forgery@prometheus@backend@prometheus>scrape-backend

Possible: server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor

Possible: server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api

Possible: server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter

Possible: unguarded-access-from-internet@backend@ingress-controller@ingress-controller>routing-backend

Possible: unencrypted-communication@backend>conexion-base-datos@backend@postgres

Improbable: cross-site-request-forgery@nginx@ingress-controller>routing-frontend

Improbable: missing-hardening@postgres

Improbable: missing-hardening@volumen-postgres

Improbable: missing-network-segmentation@postgres

Improbable: unencrypted-asset@backend

Improbable: unencrypted-asset@postgres

Improbable: unencrypted-asset@volumen-postgres

credenciales-bd: 19 / 19 Risks

Usuario y contraseña de PostgreSQL utilizados por el backend para conectarse a la base de datos.

ID:	credenciales-bd
Usage:	business
Quantity:	few
Tags:	none
Origin:	
Owner:	
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Su exposición permite acceso directo no autorizado a toda la base de datos.
Processed by:	backend, postgres
Stored by:	postgres, volumen-postgres
Sent via:	escritura_volumen, conexion_base_datos
Received via:	none
Data Breach:	probable
Data Breach Risks:	This data asset has data breach potential because of 19 remaining risks:

Probable: container-baseimage-backdooring@backend

Probable: container-baseimage-backdooring@postgres

Probable: container-baseimage-backdooring@volumen-postgres

Probable: path-traversal@postgres@volumen-postgres@postgres>escritura-volumen

Probable: sql-nosql-injection@backend@postgres@backend>conexion-base-datos

Possible: missing-authentication@ingress-controller>routing-backend@ingress-controller@backend

Possible: server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus

Possible: server-side-request-forgery@prometheus@backend@prometheus>scrape-backend

Possible: server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor

Possible: server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api

Possible: server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter

Possible: unguarded-access-from-internet@backend@ingress-controller@ingress-controller>routing-backend

Possible: unencrypted-communication@backend>conexion-base-datos@backend@postgres

Improbable: missing-hardening@postgres

Improbable: missing-hardening@volumen-postgres

Improbable: missing-network-segmentation@postgres

Improbable: unencrypted-asset@backend

Improbable: unencrypted-asset@postgres

Improbable: unencrypted-asset@volumen-postgres

credenciales-registro: 8 / 8 Risks

Usuario y token de Docker Hub almacenados como Secrets de GitHub Actions para publicar la imagen histórica del backend.

ID:	credenciales-registro
Usage:	devops
Quantity:	few
Tags:	none
Origin:	
Owner:	
Confidentiality:	strictly-confidential (rated 5 in scale of 5)
Integrity:	critical (rated 4 in scale of 5)
Availability:	operational (rated 2 in scale of 5)
CIA-Justification:	Su exposición permitiría publicar o alterar imágenes en el repositorio autorizado.
Processed by:	docker-registry, github-actions-pipeline
Stored by:	none
Sent via:	publica-backend-historico
Received via:	none

Data Breach: **probable**

Data Breach Risks: This data asset has data breach potential because of 8 remaining risks:

Probable: accidental-secret-leak@docker-registry

Probable: code-backdooring@docker-registry

Probable: code-backdooring@github-actions-pipeline

Possible: server-side-request-forgery@github-actions-pipeline@docker-registry@github-actions-pipeline>publica-backend-historico

Possible: unchecked-deployment@github-actions-pipeline

Possible: unchecked-deployment@docker-registry

Improbable: missing-hardening@github-actions-pipeline

Improbable: missing-network-segmentation@github-actions-pipeline

credenciales-usuario: 3 / 3 Risks

Usuario y contraseña de laboratorio usados por el administrador para acceder a Grafana.

ID:	credenciales-usuario	
Usage:	business	
Quantity:	few	
Tags:	none	
Origin:		
Owner:		
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Comprometer estas credenciales expone los paneles y la configuración de observabilidad.	
Processed by:	grafana	
Stored by:	none	
Sent via:	none	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 3 remaining risks:	
	Probable: container-baseimage-backdooring@grafana	
	Possible: server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus	
	Improbable: unencrypted-asset@grafana	

metadata-descubrimiento-k8s: 10 / 10 Risks

Lista de Pods y Endpoints (IP, puerto, labels) que el API Server devuelve a Prometheus en cada consulta de service discovery -- de ahí sale qué targets scrapear.

ID:	metadata-descubrimiento-k8s	
Usage:	devops	
Quantity:	few	
Tags:	none	
Origin:		
Owner:		
Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Es metadata operativa del clúster (topología de Pods/Endpoints), no datos de negocio; su alteración podría hacer que Prometheus deje de scrapear targets reales.	
Processed by:	kubernetes-api-server, prometheus	
Stored by:	none	
Sent via:	none	
Received via:	service_discovery_k8s_api	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 10 remaining risks:	

Probable: container-baseimage-backdooring@kubernetes-api-server

Probable: container-baseimage-backdooring@prometheus

Possible: cross-site-scripting@kubernetes-api-server

Possible: server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus

Possible: server-side-request-forgery@prometheus@backend@prometheus>scrape-backend

Possible: server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor

Possible: server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api

Possible: server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter

Improbable: cross-site-request-forgery@kubernetes-api-server@prometheus>service-discovery-k8s-api

Improbable: unencrypted-asset@prometheus

metricas-monitoreo: 11 / 11 Risks

Series de tiempo de métricas de aplicación, host y contenedores recolectadas por Prometheus.

ID:	metricas-monitoreo		
Usage:	devops		
Quantity:	many		
Tags:	none		
Origin:			
Owner:			
Confidentiality:	internal	(rated 2 in scale of 5)	
Integrity:	operational	(rated 2 in scale of 5)	
Availability:	operational	(rated 2 in scale of 5)	
CIA-Justification:	Son datos operativos de bajo impacto directo sobre el negocio, pero relevantes para observabilidad.		
Processed by:	cadvisor, grafana, node-exporter, prometheus		
Stored by:	prometheus		
Sent via:	none		
Received via:	scrape_node_exporter, scrape_cadvisor, scrape_backend, consulta_datos_prometheus		
Data Breach:	probable		
Data Breach Risks:	This data asset has data breach potential because of 11 remaining risks:		
	Probable: container-baseimage-backdooring@cadvisor		
	Probable: container-baseimage-backdooring@grafana		
	Probable: container-baseimage-backdooring@node-exporter		
	Probable: container-baseimage-backdooring@prometheus		
	Possible: server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus		
	Possible: server-side-request-forgery@prometheus@backend@prometheus>scrape-backend		
	Possible: server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor		
	Possible: server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api		
	Possible: server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter		
	Improbable: unencrypted-asset@grafana		
	Improbable: unencrypted-asset@prometheus		

token-service-account-prometheus: 7 / 7 Risks

Token de la ServiceAccount "prometheus-sa" que autentica a Prometheus contra el API Server de Kubernetes para el service discovery dinámico (RBAC vía ClusterRole/ClusterRoleBinding, de solo lectura sobre pods/services/endpoints/nodes).

ID:	token-service-account-prometheus	
Usage:	devops	
Quantity:	few	
Tags:	none	
Origin:		
Owner:		
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Si se filtra, permite listar pods, services, endpoints y nodes de todo el clúster (alcance ClusterRole, no sólo del namespace).	
Processed by:	prometheus	
Stored by:	none	
Sent via:	service_discovery_k8s_api	
Received via:	none	
Data Breach:	probable	
Data Breach Risks:	This data asset has data breach potential because of 7 remaining risks:	
	Probable: container-baseimage-backdooring@prometheus	
	Possible: server-side-request-forgery@grafana@prometheus@grafana>consulta-datos-prometheus	
	Possible: server-side-request-forgery@prometheus@backend@prometheus>scrape-backend	
	Possible: server-side-request-forgery@prometheus@cadvisor@prometheus>scrape-cadvisor	
	Possible: server-side-request-forgery@prometheus@kubernetes-api-server@prometheus>service-discovery-k8s-api	
	Possible: server-side-request-forgery@prometheus@node-exporter@prometheus>scrape-node-exporter	
	Improbable: unencrypted-asset@prometheus	

Trust Boundaries

In total **1 trust boundaries** has been modeled during the threat modeling process.

cluster-kubernetes

Clúster k3d local completo; agrupa los contenedores y el volumen para evaluar riesgos en las fronteras de red.

ID:	cluster-kubernetes
Type:	network-on-prem
Tags:	none
Assets inside:	backend, cadvisor, grafana, ingress-controller, kubernetes-api-server, nginx, node-exporter, postgres, prometheus, volumen-postgres
Boundaries nested:	none

Shared Runtimes

In total **0 shared runtime** has been modeled during the threat modeling process.

Risk Rules Checked by Threagile

Threagile Version: 1.0.0

Threagile Build Timestamp: 20240730113903

Threagile Execution Timestamp: 20260916180056

Model Filename: /app/work/threagile.yaml

Model Hash (SHA256): fce0e62c89842695fe7eea24d3279db16623e114ccc3bbc85aced59dcc2c03c1

Threagile (see <https://threagile.io> for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (<https://christian-schneider.net>): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Accidental Secret Leak

accidental-secret-leak

STRIDE: Information Disclosure

Description: Sourcecode repositories (including their histories) as well as artifact registries can accidentally contain secrets like checked-in or packaged-in passwords, API tokens, certificates, crypto keys, etc.

Detection: In-scope sourcecode repositories and artifact registries.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Code Backdooring

code-backdooring

STRIDE: Tampering

Description: For each build-pipeline component Code Backdooring risks might arise where attackers compromise the build-pipeline in order to let backdoored artifacts be shipped into production. Aside from direct code backdooring this includes backdooring of dependencies and even of more lower-level build infrastructure, like backdooring compilers (similar to what the XcodeGhost malware did) or dependencies.

Detection: In-scope development relevant technical assets which are either accessed by out-of-scope unmanaged developer clients and/or are directly accessed by any kind of internet-located (non-VPN) component or are themselves directly located on the internet.

Rating: The risk rating depends on the confidentiality and integrity rating of the code being handled and deployed as well as the placement/calling of this technical asset on/from the internet.

Container Base Image Backdooring

container-baseimage-backdooring

STRIDE: Tampering

Description: When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

Detection: In-scope technical assets running as containers.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets.

Container Platform Escape

container-platform-escape

STRIDE: Elevation of Privilege

Description: Container platforms are especially interesting targets for attackers as they host big parts of a containerized runtime infrastructure. When not configured and operated with security best practices in mind, attackers might exploit a vulnerability inside an container and escape towards the platform as highly privileged users. These scenarios might give attackers capabilities to attack every other container as owning the container platform (via container escape attacks) equals to owning every container.

Detection: In-scope container platforms.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Cross-Site Request Forgery (CSRF)

cross-site-request-forgery

STRIDE: Spoofing

Description: When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.

Detection: In-scope web applications accessed via typical web access protocols.

Rating: The risk rating depends on the integrity rating of the data sent across the communication link.

Cross-Site Scripting (XSS)

cross-site-scripting

STRIDE: Tampering

Description: For each web application Cross-Site Scripting (XSS) risks might arise. In terms of the overall risk level take other applications running on the same domain into account as well.

Detection: In-scope web applications.

Rating: The risk rating depends on the sensitivity of the data processed or stored in the web application.

DoS-risky Access Across Trust-Boundary

dos-risky-access-across-trust-boundary

STRIDE: Denial of Service

Description: Assets accessed across trust boundaries with critical or mission-critical availability rating are more prone to Denial-of-Service (DoS) risks.

Detection: In-scope technical assets (excluding load-balancer) with availability rating of critical or higher which have incoming data-flows across a network trust-boundary (excluding devops usage).

Rating: Matching technical assets with availability rating of critical or higher are at low risk. When the availability rating is mission-critical and neither a VPN nor IP filter for the incoming data-flow nor redundancy for the asset is applied, the risk-rating is considered medium.

Incomplete Model**incomplete-model**

STRIDE: Information Disclosure

Description: When the threat model contains unknown technologies or transfers data over unknown protocols, this is an indicator for an incomplete model.

Detection: All technical assets and communication links with technology type or protocol type specified as unknown.

Rating: low

LDAP-Injection**ldap-injection**

STRIDE: Tampering

Description: When an LDAP server is accessed LDAP-Injection risks might arise. The risk rating depends on the sensitivity of the LDAP server itself and of the data assets processed or stored.

Detection: In-scope clients accessing LDAP servers via typical LDAP access protocols.

Rating: The risk rating depends on the sensitivity of the LDAP server itself and of the data assets processed or stored.

Missing Authentication**missing-authentication**

STRIDE: Elevation of Privilege

Description: Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes or stores sensitive data.

Detection: In-scope technical assets (except load-balancer, reverse-proxy, service-registry, waf, ids, and ips and in-process calls) should authenticate incoming requests when the asset processes or stores sensitive data. This is especially the case for all multi-tenant assets (there even non-sensitive ones).

Rating: The risk rating (medium or high) depends on the sensitivity of the data sent across

the communication link. Monitoring callers are exempted from this risk.

Missing Two-Factor Authentication (2FA)

missing-authentication-second-factor

STRIDE: Elevation of Privilege

Description: Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Detection: In-scope technical assets (except load-balancer, reverse-proxy, waf, ids, and ips) should authenticate incoming requests via two-factor authentication (2FA) when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by a client used by a human user.

Rating: medium

Missing Build Infrastructure

missing-build-infrastructure

STRIDE: Tampering

Description: The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Detection: Models with in-scope custom-developed parts missing in-scope development (code creation) and build infrastructure components (devops-client, sourcecode-repo, build-pipeline, etc.).

Rating: The risk rating depends on the highest sensitivity of the in-scope assets running custom-developed parts.

Missing Cloud Hardening

missing-cloud-hardening

STRIDE: Tampering

Description: Cloud components should be hardened according to the cloud vendor best practices. This affects their configuration, auditing, and further areas.

Detection: In-scope cloud components (either residing in cloud trust boundaries or more specifically tagged with cloud provider types).

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Missing File Validation

missing-file-validation

STRIDE: Spoofing

- Description: When a technical asset accepts files, these input files should be strictly validated about filename and type.
- Detection: In-scope technical assets with custom-developed code accepting file data formats.
- Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Missing Hardening

missing-hardening

- STRIDE: Tampering
- Description: Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.
- Detection: In-scope technical assets with RAA values of 55 % or higher. Generally for high-value targets like datastores, application servers, identity providers and ERP systems this limit is reduced to 40 %
- Rating: The risk rating depends on the sensitivity of the data processed or stored in the technical asset.

Missing Identity Propagation

missing-identity-propagation

- STRIDE: Elevation of Privilege
- Description: Technical assets (especially multi-tenant systems), which usually process data for endusers should authorize every request based on the identity of the enduser when the data flow is authenticated (i.e. non-public). For DevOps usages at least a technical-user authorization is required.
- Detection: In-scope service-like technical assets which usually process data based on enduser requests, if authenticated (i.e. non-public), should authorize incoming requests based on the propagated enduser identity when their rating is sensitive. This is especially the case for all multi-tenant assets (there even less-sensitive rated ones). DevOps usages are exempted from this risk.
- Rating: The risk rating (medium or high) depends on the confidentiality, integrity, and availability rating of the technical asset.

Missing Identity Provider Isolation

missing-identity-provider-isolation

- STRIDE: Elevation of Privilege
- Description: Highly sensitive identity provider assets and their identity datastores should be isolated from other assets by their own network segmentation trust-boundary (execution-environment boundaries do not count as network isolation).
- Detection: In-scope identity provider assets and their identity datastores when surrounded by other (not identity-related) assets (without a network trust-boundary in-between).

This risk is especially prevalent when other non-identity related assets are within the same execution environment (i.e. same database or same application server).

Rating: Default is high impact. The impact is increased to very-high when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

Missing Identity Store

missing-identity-store

STRIDE: Spoofing

Description: The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Detection: Models with authenticated data-flows authorized via enduser-identity missing an in-scope identity store.

Rating: The risk rating depends on the sensitivity of the enduser-identity authorized technical assets and their data assets processed and stored.

Missing Network Segmentation

missing-network-segmentation

STRIDE: Elevation of Privilege

Description: Highly sensitive assets and/or datastores residing in the same network segment than other lower sensitive assets (like webserver or content management systems etc.) should be better protected by a network segmentation trust-boundary.

Detection: In-scope technical assets with high sensitivity and RAA values as well as datastores when surrounded by assets (without a network trust-boundary in-between) which are of type client-system, web-server, web-application, cms, web-service-rest, web-service-soap, build-pipeline, sourcecode-repository, monitoring, or similar and there is no direct connection between these (hence no requirement to be so close to each other).

Rating: Default is low risk. The risk is increased to medium when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

Missing Vault (Secret Storage)

missing-vault

STRIDE: Information Disclosure

Description: In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Detection: Models without a Vault (Secret Storage).

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Missing Vault Isolation

missing-vault-isolation

STRIDE: Elevation of Privilege

Description: Highly sensitive vault assets and their datastores should be isolated from other assets by their own network segmentation trust-boundary (execution-environment boundaries do not count as network isolation).

Detection: In-scope vault assets when surrounded by other (not vault-related) assets (without a network trust-boundary in-between). This risk is especially prevalent when other non-vault related assets are within the same execution environment (i.e. same database or same application server).

Rating: Default is medium impact. The impact is increased to high when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

Missing Web Application Firewall (WAF)

missing-waf

STRIDE: Tampering

Description: To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Detection: In-scope web-services and/or web-applications accessed across a network trust boundary not having a Web Application Firewall (WAF) in front of them.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Mixed Targets on Shared Runtime

mixed-targets-on-shared-runtime

STRIDE: Elevation of Privilege

Description: Different attacker targets (like frontend and backend/datastore components) should not be running on the same shared (underlying) runtime.

Detection: Shared runtime running technical assets of different trust-boundaries is at risk. Also mixing backend/datastore with frontend components on the same shared runtime is considered a risk.

Rating: The risk rating (low or medium) depends on the confidentiality, integrity, and availability rating of the technical asset running on the shared runtime.

Path-Traversal

path-traversal

STRIDE: Information Disclosure

Description: When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself

and of the data assets processed or stored.

Detection: Filesystems accessed by in-scope callers.

Rating: The risk rating depends on the sensitivity of the data stored inside the technical asset.

Push instead of Pull Deployment

push-instead-of-pull-deployment

STRIDE: Tampering

Description: When comparing push-based vs. pull-based deployments from a security perspective, pull-based deployments improve the overall security of the deployment targets. Every exposed interface of a production system to accept a deployment increases the attack surface of the production system, thus a pull-based approach exposes less attack surface relevant interfaces.

Detection: Models with build pipeline components accessing in-scope targets of deployment (in a non-readonly way) which are not build-related components themselves.

Rating: The risk rating depends on the highest sensitivity of the deployment targets running custom-developed parts.

Search-Query Injection

search-query-injection

STRIDE: Tampering

Description: When a search engine server is accessed Search-Query Injection risks might arise.

Detection: In-scope clients accessing search engine servers via typical search access protocols.

Rating: The risk rating depends on the sensitivity of the search engine server itself and of the data assets processed or stored.

Server-Side Request Forgery (SSRF)

server-side-request-forgery

STRIDE: Information Disclosure

Description: When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Detection: In-scope non-client systems accessing (using outgoing communication links) targets with either HTTP or HTTPS protocol.

Rating: The risk rating (low or medium) depends on the sensitivity of the data assets receivable via web protocols from targets within the same network trust-boundary as well on the sensitivity of the data assets receivable via web protocols from the target asset itself. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF.

Service Registry Poisoning

service-registry-poisoning

STRIDE: Spoofing

Description: When a service registry used for discovery of trusted service endpoints Service Registry Poisoning risks might arise.

Detection: In-scope service registries.

Rating: The risk rating depends on the sensitivity of the technical assets accessing the service registry as well as the data assets processed or stored.

SQL/NoSQL-Injection

sql-nosql-injection

STRIDE: Tampering

Description: When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed or stored.

Detection: Database accessed via typical database access protocols by in-scope clients.

Rating: The risk rating depends on the sensitivity of the data stored inside the database.

Unchecked Deployment

unchecked-deployment

STRIDE: Tampering

Description: For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Detection: All development-relevant technical assets.

Rating: The risk rating depends on the highest rating of the technical assets and data assets processed by deployment-receiving targets.

Unencrypted Technical Assets

unencrypted-asset

STRIDE: Information Disclosure

Description: Due to the confidentiality rating of the technical asset itself and/or the processed data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Detection: In-scope unencrypted technical assets (excluding reverse-proxy, load-balancer, waf, ids, ips and embedded components like library) storing data assets rated at least as confidential or critical. For technical assets storing data assets rated as strictly-confidential or mission-critical the encryption must be of type data-with-enduser-individual-key.

Rating: Depending on the confidentiality rating of the stored data-assets either medium or high risk.

Unencrypted Communication

unencrypted-communication

STRIDE: Information Disclosure

Description: Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Detection: Unencrypted technical communication links of in-scope technical assets (excluding monitoring traffic as well as local-file-access and in-process-library-call) transferring sensitive data.

Rating: Depending on the confidentiality rating of the transferred data-assets either medium or high risk.

Unguarded Access From Internet

unguarded-access-from-internet

STRIDE: Elevation of Privilege

Description: Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Detection: In-scope technical assets (excluding load-balancer) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) when accessed directly from the internet. All web-server, web-application, reverse-proxy, waf, and gateway assets are exempted from this risk when they do not consist of custom developed code and the data-flow only consists of HTTP or FTP protocols. Access from monitoring systems as well as VPN-protected connections are exempted.

Rating: The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

Unguarded Direct Datastore Access

unguarded-direct-datastore-access

STRIDE: Elevation of Privilege

Description: Datastores accessed across trust boundaries must be guarded by some protecting service or application.

Detection: In-scope technical assets of type datastore (except identity-store-ldap when accessed from identity-provider and file-server when accessed via file transfer protocols) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) which have incoming data-flows from assets outside across a network trust-boundary. DevOps config and deployment access is excluded from this risk.

Rating: The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

Unnecessary Communication Link

unnecessary-communication-link

STRIDE: Elevation of Privilege
Description: When a technical communication link does not send or receive any data assets, this is an indicator for an unnecessary communication link (or for an incomplete model).
Detection: In-scope technical assets' technical communication links not sending or receiving any data assets.
Rating: low

Unnecessary Data Asset

unnecessary-data-asset

STRIDE: Elevation of Privilege
Description: When a data asset is not processed or stored by any data assets and also not transferred by any communication links, this is an indicator for an unnecessary data asset (or for an incomplete model).
Detection: Modelled data assets not processed or stored by any data assets and also not transferred by any communication links.
Rating: low

Unnecessary Data Transfer

unnecessary-data-transfer

STRIDE: Elevation of Privilege
Description: When a technical asset sends or receives data assets, which it neither processes or stores this is an indicator for unnecessarily transferred data (or for an incomplete model). When the unnecessarily transferred data assets are sensitive, this poses an unnecessary risk of an increased attack surface.
Detection: In-scope technical assets sending or receiving sensitive data assets which are neither processed nor stored by the technical asset are flagged with this risk. The risk rating (low or medium) depends on the confidentiality, integrity, and availability rating of the technical asset. Monitoring data is exempted from this risk.
Rating: The risk assessment is depending on the confidentiality and integrity rating of the transferred data asset either low or medium.

Unnecessary Technical Asset

unnecessary-technical-asset

STRIDE: Elevation of Privilege
Description: When a technical asset does not process or store any data assets, this is an

indicator for an unnecessary technical asset (or for an incomplete model). This is also the case if the asset has no communication links (either outgoing or incoming).

Detection: Technical assets not processing or storing any data assets.

Rating: low

Untrusted Deserialization

untrusted-deserialization

STRIDE: Tampering

Description: When a technical asset accepts data in a specific serialized form (like Java or .NET serialization), Untrusted Deserialization risks might arise.

Detection: In-scope technical assets accepting serialization data formats (including EJB and RMI protocols).

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Wrong Communication Link Content

wrong-communication-link-content

STRIDE: Information Disclosure

Description: When a communication link is defined as readonly, but does not receive any data asset, or when it is defined as not readonly, but does not send any data asset, it is likely to be a model failure.

Detection: Communication links with inconsistent data assets being sent/received not matching their readonly flag or otherwise inconsistent protocols not matching the target technology type.

Rating: low

Wrong Trust Boundary Content

wrong-trust-boundary-content

STRIDE: Elevation of Privilege

Description: When a trust boundary of type network-policy-namespace-isolation contains non-container assets it is likely to be a model failure.

Detection: Trust boundaries which should only contain containers, but have different assets inside.

Rating: low

XML External Entity (XXE)

xml-external-entity

STRIDE: Information Disclosure

Description: When a technical asset accepts data in XML format, XML External Entity (XXE) risks might arise.

Detection: In-scope technical assets accepting XML data formats.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data

assets processed and stored. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF (and XXE vulnerabilities are often also SSRF vulnerabilities).

Disclaimer

Equipo DevOps - Notes App conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. Equipo DevOps - Notes App and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that Equipo DevOps - Notes App and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a timebox approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.